Practical 1

In [61]:
```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, cross_val_predict, KFold
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, mean_squared_error, r2_score
from sklearn.preprocessing import label_binarize
```

In [62]:
```
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

In [63]:
```
# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize and fit the Naïve Bayes classifier
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)
# Predict on the test set
y_pred = nb_classifier.predict(X_test)
```

In [64]:
```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = conf_matrix.ravel() if conf_matrix.size == 4 else (None, None, None, None)
# Classification Report for Precision, Recall, and F1-Score
class_report = classification_report(y_test, y_pred)
```

In [65]:
```
# AUC score (needs binary classification, here we binarize for multi-class ROC-AUC)
y_bin = label_binarize(y, classes=[0, 1, 2])
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
y_pred_proba = nb_classifier.predict_proba(X_test)
auc = roc_auc_score(y_test_bin, y_pred_proba, multi_class="ovr")
print(f"Accuracy: {accuracy}")
```

```
print("Confusion Matrix:\n", conf_matrix)
print("Classification Report:\n", class_report)
print(f"AUC Score: {auc}")
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

AUC Score: 1.0
```

In [66]:
```
# 10-fold cross-validation
kfold = KFold(n_splits=10, random_state=42, shuffle=True)
cv_accuracy = cross_val_score(nb_classifier, X, y, cv=kfold, scoring="accuracy")
print(f"Cross-Validation Accuracy: {cv_accuracy.mean()}")
```

```
Cross-Validation Accuracy: 0.9600000000000002
```

In [ ]:
```

```